

基于多场景用户需求的大语言模型个性化推荐研究

队伍：26144

摘 要

随着大语言模型市场的快速扩张，模型数量激增、定价机制多样、能力维度分化，用户在选择 AI 工具时面临难以兼顾能力匹配与成本控制的现实困境。针对这一问题，本研究构建了一套通过客观能力评价再到主观场景适配和资源配置优化的两阶段联合推荐框架。

在研究工作中，我们首先挑选了 6 款主流大语言模型，并从编程、Agent、长上下文、知识推理、专业推理、成本价格与生成速度七个维度构建评价指标体系，预处理后采用熵权-TOPSIS 方法计算各模型的分维度能力贴近度，形成客观能力矩阵；随后针对六类典型用户群体，采用层次分析法确定各群体的场景化偏好权重，将能力矩阵、任务需求与偏好权重融合为综合效用函数，并结合 API 调用成本与预算约束，以 0-1 整数规划求解全局最优的模型-任务分配方案。

研究表明，DeepSeek V4 Pro 在成本敏感型场景中占据绝对优势，Claude Opus 4.8 在能力驱动型场景中表现突出，GPT-5.5 在学术研究场景中排名第一，验证了多模型混合调用优于单一模型的资源配置规律。本文提出的框架将大语言模型选型从经验驱动转化为数据驱动的组合优化决策，为不同用户群体提供了可落地的选型与任务分配方案，具有良好的可扩展性与推广价值。

关键词：大语言模型、熵权-TOPSIS、AHP 层次分析法、0-1 整数规划

目 录

摘要.....	I
第一章 引言	1
1.1 研究背景	1
1.2 相关研究综述	2
1.2.1 大语言模型评测相关研究	2
1.2.2 大语言模型定价与成本优化相关研究	2
1.2.3 大语言模型推荐与资源分配相关研究	2
1.3 研究意义与创新点	2
1.3.1 研究意义	2
1.3.2 创新点	3
第二章 模型总体思路	4
2.1 建模基本方法	4
2.2 基本假设	5
2.3 符号表	6
第三章 基于熵权-TOPSIS 的大语言模型能力画像	7
3.1 指标体系构建	7
3.1.1 评价对象	7
3.1.2 评价指标	7
3.2 指标预处理	9
3.2.1 指标的一致化处理	9
3.2.2 无量纲化处理	10
3.3 基于熵权法的客观权重计算	10
3.4 基于 TOPSIS 的各模型多维能力得分矩阵构建	11
3.5 综合评价模型求解及分析	12
3.5.1 数据收集及预处理结果	12
3.5.2 熵权法结果	13
3.5.3 分维度能力画像	15
第四章 融合主观场景与预算约束的个性化资源配置优化模型	16
4.1 用户场景划分与任务需求解构	16
4.1.1 典型用户场景分类	16
4.1.2 任务需求矩阵估计	16

4.2 基于 AHP 的用户场景主观偏好权重确定	17
4.2.1 层次结构模型搭建	17
4.2.2 判断矩阵构建	18
4.2.3 偏好权重计算与一致性检验	18
4.3 基于 0-1 整数规划的资源配置模型构建	19
4.3.1 决策变量定义	19
4.3.2 综合效用函数构建	19
4.3.3 成本函数构建	20
4.3.4 目标函数与约束条件	20
4.4 推荐结果与场景分析	20
4.4.1 用户偏好权重结果分析	20
4.4.2 六类用户多任务的推荐结果	22
第五章 总结与展望	24
5.1 总结	24
5.2 模型优缺点	24
5.2.1 模型优点	24
5.2.2 模型缺点	24
5.3 展望	25
参考文献	26
附 录	27

第一章 引言

1.1 研究背景

随着生成式人工智能的爆发式发展，大语言模型市场正经历前所未有的扩张。从 GPT 系列、Claude 系列到 DeepSeek、Kimi、GLM 等国产大模型，各类大语言模型在推理能力、代码生成、长文本处理、多模态理解等维度上形成了差异化竞争格局。与此同时，各厂商推出了多样化的商业化购买方案，包括按 token 计费的 API 调用、订阅制套餐、免费额度与赠送 token 策略、企业批量折扣等——使得定价机制日趋复杂，且价格调整频繁，市场呈现高度非平稳的动态博弈特征。



图 1.1: 常见大模型

在此背景下，轻度用户、普通用户、软件开发者、学术研究者、企业团队及 AI 创业者等不同群体面临着日益突出的选择上的困难。不同用户的使用频率、任务类型、预算约束与性能偏好存在显著差异：一名轻度用户可能仅需处理日常问答，对成本高度敏感；而一名科研工作者则可能频繁调用长文本推理任务，更看重模型的理解深度与上下文窗口，对价格敏感度相对较低。然而，现有研究及第三方评测平台通常侧重于静态的模型性能排名或单一维度的性价比比较，对于不同用户需求差异、实际应用场景变化以及多模型组合使用所带来的成本—性能帕累托优化空间关注不足。

更为关键的是，大语言模型市场的价格与性能均随时间快速演化——新模型不断发布、老模型持续降价、免费额度定期更新——静态评估结果往往在短期内即失去参考价值。因此，用户需要一种能够结合自身需求特征、适应模型市场变化，并支持多产品组合选择的动态推荐框架，从而在能力、价格与速度之间实现更合理的权衡。

本研究目的是针对上述问题，构建一套面向用户需求的大语言模型推荐方法，使用户能够结合自身任务特点与实际使用场景，在复杂多变的市场环境中选择更适合的模型与服务组合，从而提升工作与学习效率。

1.2 相关研究综述

1.2.1 大语言模型评测相关研究

大语言模型评测是模型选择与应用的基础。现有研究主要围绕模型能力、安全性与可靠性等维度展开。Huzhi Xue 等人针对传统评测体系覆盖有限、评价维度单一的问题，构建了涵盖偏差、服从性与鲁棒性等六项指标的综合评估框架，实现了模型性能的量化排序 [1]。然而，该研究在一定程度上依赖人工标注，存在较强主观性，且未纳入调用成本、速率限制及上下文容量等实际使用因素，导致评估结果与用户真实选型需求之间仍存在差距。

1.2.2 大语言模型定价与成本优化相关研究

随着大语言模型商业化程度不断提高，其定价机制逐渐成为研究热点。Zhong 从信息设计视角分析了基于用户时间敏感性的最优定价策略 [2]；Bergemann 等从合同理论角度构建了 token 用量分层定价框架 [3]；Mahmood 则从市场竞争角度揭示了先发者与后发者在成本效益上的博弈关系 [4]。现有研究从平台收益优化、合同设计与竞争机制等角度系统分析了 LLM 定价问题，但多数研究仍主要面向服务提供商视角，缺乏从用户资源配置角度进行综合成本建模的研究。

1.2.3 大语言模型推荐与资源分配相关研究

近年来，大语言模型推荐与资源分配逐渐受到关注。Lingjiao Chen 等提出了多模型级联调用框架，以降低整体推理成本 [5]；Ong 等进一步实现了提示词级别的动态路由，提高了模型选择效率 [6]。然而，现有方法大多建立在需求规模、预算约束和调用成本完全确定的静态假设之上，未充分考虑用户 token 消耗波动、免费额度到期及预算超支风险等不确定因素。因此，在多场景、多约束条件下的大模型全局资源优化配置问题仍有待进一步研究。

然而，现有研究在多维评价指标融合与用户异质性建模方面仍存在割裂，尚未形成面向真实使用场景的统一决策框架。基于此，本文拟构建面向多场景用户需求的大语言模型综合评价体系，并进一步建立考虑不确定性的个性化推荐优化模型，以实现能力—成本—效率之间的协同最优。

1.3 研究意义与创新点

1.3.1 研究意义

- **理论意义：**现有研究多将大语言模型选择简化为性能排名问题，本文将其重新定义为预算约束下的多目标组合优化，体现了模型能力差异与用户任务需求之间的匹配关系，并验证了多模型混合调用在资源配置上优于单一模型的规律。

- **模型意义：**本文将熵权-TOPSIS、AHP 与 0-1 整数规划整合为多层建模框架，使推荐逻辑从“哪个模型最好”转变为“在预算内如何组合模型最合适”，让静态的能力评分真正服务于动态的决策需求。
- **现实意义：**面对模型种类繁多、定价复杂、能力参差不齐的市场现状，普通用户的选型成本正在快速上升。本研究提供了一套从能力评价到个性化推荐的完整方法，帮助不同背景的用户找到真正适合自己场景的模型组合方案。

1.3.2 创新点

- **构建多维能力评价体系**

本文使用 TOPSIS 方法建立模型能力评价矩阵，从七个不同方面分析模型能力，并根据不同任务的需求选择更合适的模型。

- **融合能力评价、用户偏好与资源优化**

我们将模型能力评价、用户偏好分析和资源分配放在同一个框架中，分析不同模型特点与用户需求之间的关系，并通过优化方法解决模型选择问题。

- **面向多类用户构建场景化个性推荐机制**

此研究针对六类常见用户，分别分析他们对模型的不同需求，并计算对应的指标权重，从而让模型推荐过程能够根据用户特点进行调整。

第二章 模型总体思路

2.1 建模基本方法

本文针对大语言模型个性化推荐问题，构建了“客观能力评价—主观场景适配与资源配置优化”的两阶段联合建模框架，如图2.1所示。

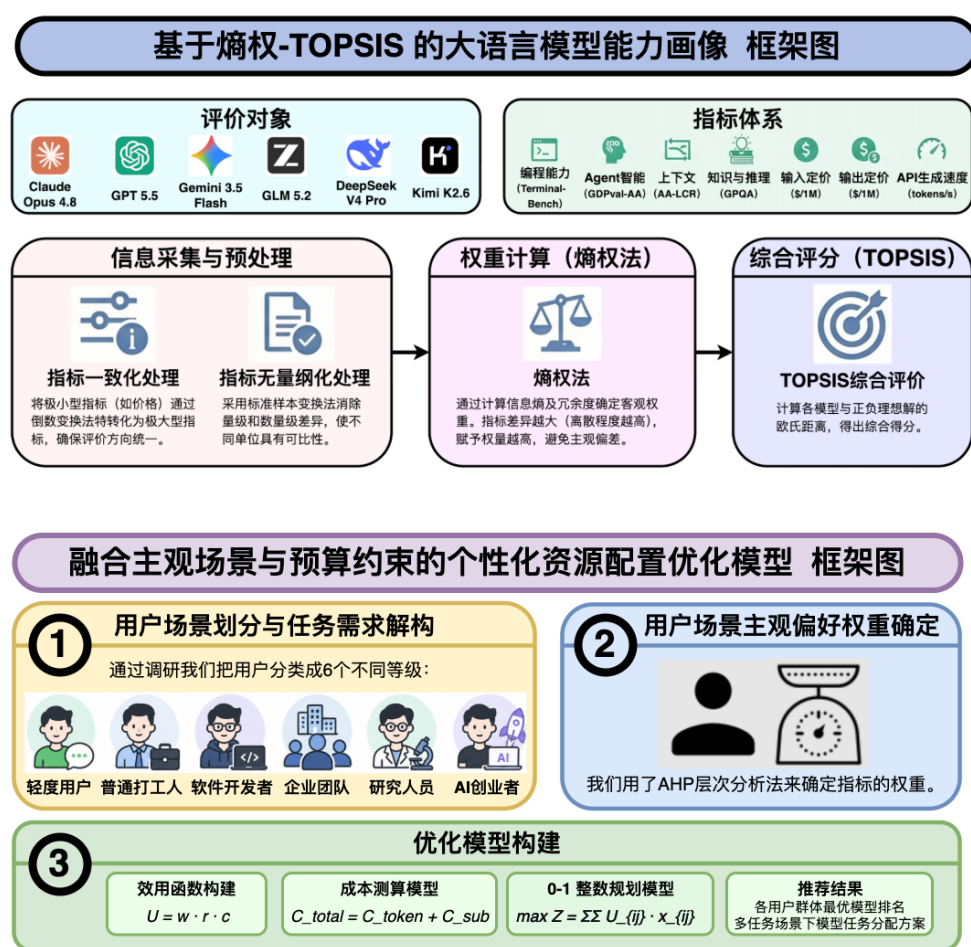


图 2.1: 模型整体框架图

第一阶段基于熵权-TOPSIS 方法，构建六款主流大语言模型在编程能力、Agent 能力、长上下文、知识推理、专业推理、成本价格与生成速度七个维度上的客观能力矩阵；第二阶段引入 AHP 层次分析法确定六类典型用户群体的场景化偏好权重，将能力矩阵与偏好权重、任务需求融合为效用函数，并结合 token 使用成本与预算约束，以 0-1 整数规划求解全局最优的模型-任务配置方案。两阶段模型实现了从客观能力评价到个性化推荐决策的完整闭环。

2.2 基本假设

为保证模型构建与求解过程具有可行性，并在一定程度上降低现实环境中复杂因素对研究的干扰，本文结合大语言模型市场特征及用户使用行为，提出以下基本假设：

- **评价有效性假设**

假设各公开 Benchmark 测试结果在一定程度上能够反映大语言模型的相对能力水平，并在研究周期内保持可比性，且评测标准在研究周期内保持一致，不存在显著的外部扰动。

- **用户需求稳定性假设**

假设用户在一定决策周期内的任务类型、使用频率及预算约束相对稳定，可抽象为若干具有代表性的典型用户画像，且各画像在研究窗口内的核心需求任务偏好以及预算约束结构保持相对稳定，从而便于进行需求建模与个性化推荐分析。

- **成本可加性假设**

假设用户使用大语言模型所产生的总成本由 API 调用成本、订阅成本及隐性时间成本线性叠加构成，各项成本之间相互独立，不考虑复杂的交叉补贴或动态折扣机制。

- **市场短期稳定性假设**

假设研究周期内各模型的定价策略、性能水平及用户偏好变化具有短期稳定性，不发生大规模结构性调整，从而保证基于历史数据构建的推荐模型具有现实参考意义。

- **场景独立性假设**

假设不同用户场景之间的需求偏好相互独立，各场景下模型效用可单独评估，并通过统一优化框架进行资源配置。

- **任务需求独立且可分假设**

假设同一用户的多个任务（如代码生成、文档摘要、日常问答等）在能力需求上相互独立，可分别分配给不同模型执行，且各任务对模型的能力诉求不产生相互干扰。该假设确保总成本函数具有可加性，便于采用线性规划方法求解最优分配方案。

- **预算硬约束假设**

假设用户月度预算是不可突破的刚性上限。即用户在模型选择过程中必须严格满足成本约束条件，推荐方案的总成本不得超过用户预算。该假设保证推荐结果在实际使用中具备财务可行性，真实反映用户的支付能力限制。

• API 调用成本主导假设

本文仅采用各模型官方 API 的按量计费价格进行成本核算，不考虑订阅制套餐。相同的订阅费用在不同服务中存在不一样的额度限制，没有明确的官方限额数据，因此难以横向对比。

2.3 符号表

本文中所使用的主要符号、变量及其释义总结见表2.1：

表 2.1: 主要符号说明

符号	含义
$A = (a_{ij})_{m \times n}$	原始评价矩阵
a_{ij}	第 i 个模型在第 j 个指标上的原始取值
x_{ij}	标准化后的指标值
e_j	第 j 个指标的信息熵
d_j	第 j 个指标的信息冗余度
w_j	第 j 个指标的熵权权重
V^+, V^-	正理想解与负理想解
D_i^+, D_i^-	第 i 个模型到正/负理想解的距离
C_{ik}	模型 i 在能力维度 k 上的 TOPSIS 贴近度
$\mathbf{A} = (a_{ij})_{7 \times 7}$	AHP 准则层判断矩阵
$\lambda_{\max}$	判断矩阵最大特征值
$\mathbf{w}^{\text{AHP}}$	AHP 场景化偏好权重向量（准则层）
$\mathcal{J} = \{T_1, \dots, T_M\}$	用户任务集合
M_i	第 i 个候选大语言模型
U_{ij}	模型 M_i 执行任务 T_j 的综合效用值
x_{ij}	0-1 决策变量：1 表示将模型 i 分配给任务 j
C_{ij}	模型 M_i 完成任务 T_j 的单次 Token 调用成本
f_j	任务 T_j 的月度调用次数
B	用户月度预算硬约束上限

第三章 基于熵权-TOPSIS 的大语言模型能力画像

在多场景用户需求下，大语言模型的选择需要综合考虑能力、成本与效率等多维因素。为此，本章选取六种主流大语言模型构建综合评价体系，并结合熵权法与 TOPSIS 模型计算其基础能力得分，为后续个性化推荐优化提供支撑。

3.1 指标体系构建

3.1.1 评价对象

本研究选择了六款具有行业代表性的前沿大语言模型作为评价对象，涵盖了国内外顶尖人工智能实验室的开源和闭源模型，具体见表3.1：

表 3.1: 评价对象

模型名称	版本	研发公司
Claude	Opus 4.8	Anthropic
ChatGPT	5.5	OpenAI
Gemini	3.5 Flash	Google
GLM	5.2	ZhipuAI
DeepSeek	V4 Pro	深度求索
Kimi	K2.6	MoonshotAI

3.1.2 评价指标

为全面衡量不同大语言模型在实际应用中的综合表现，本文结合现有主流评测体系以及用户实际使用场景，参考 Artificial Analysis 给出的评测框架 [7]，从编程、Agent 能力、长上下文处理、知识面与推理能力、专业领域推理、价格以及速度这七个维度构建评价体系 3.1，具体如下：

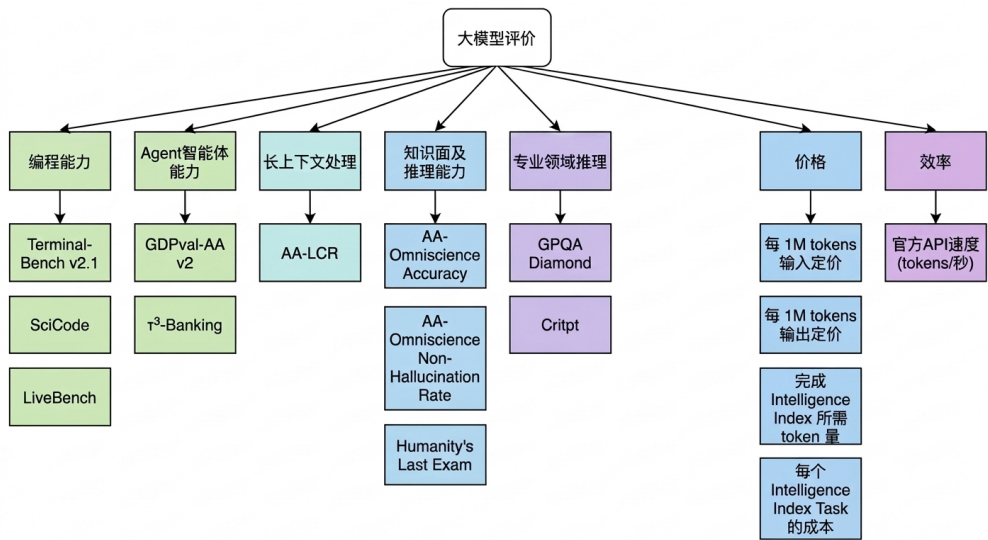


图 3.1: 评价体系

(1) **编程能力**: 反映模型在代码生成、调试与复杂工程任务中的综合表现, 是衡量模型技术应用能力的重要指标。本文选取以下三个 benchmark:

- Terminal-Bench v2.1: 评估智能体在真实 Linux 命令行环境下的复杂任务执行能力, 最终以任务预设的自动单元测试通过率计算得分。
- SciCode: 针对科学计算编程能力的基准。要求模型编写代码解决物理、数学等领域的复杂科学问题, 采用逐步引导的链式格式, 并基于模型生成代码在基准测试用例上的实际执行正确率进行评分。
- LiveBench: 一个实时更新的、防数据污染的编程与推理基准。其通过每月从最新的竞赛平台中动态抽取未公开的编程新题来测试模型, 以防范预训练数据污染, 最终按测试集通过率计分。

(2) **智能体能力**: 用于衡量模型在复杂 workflows 中的自主规划、工具调用及多轮任务执行能力, 是未来智能体应用的重要基础。

- GDPval-AA v2: 评估模型在金融、法律等 9 个美国主要经济部门中高价现实工作流的代行能力。模型需使用浏览器或 API 检索并解决综合型业务, 得分基于任务完成度及输出决策的专家标准匹配度计算。
- τ^3 -Banking: 一个针对银行和金融服务场景的特定领域智能体基准。模拟复杂的银行后台系统、CRM 和账单查询多步交互流, 通过评估智能体满足客户复杂金融业务请求的端到端成功率来打分。

(3) **长上下文处理**: 衡量模型在大规模上下文输入条件下的信息检索与推理能力, 是科研分析、代码仓库理解及长文档处理场景中的关键指标。

- AA-LCR: 大语言模型的长上下文检索测试。通过在多达数百万字的长文本的不同深处埋入关键信息, 测试模型在海量上下文中精确提取或推理该信息的能力。

(4) **知识面及推理能力**: 该维度衡量模型在跨领域知识覆盖、事实准确性以及不确定场景下的回答可靠性, 是评价模型知识广度与校准能力的重要指标。

- AA-Omniscience Accuracy: 该测试包含 6,000 道覆盖商业、健康、法律、软件工程等 6 大领域 42 个经济相关主题的高难度知识题, 要求模型具备跨领域的精确事实记忆。
- AA-Omniscience Non-Hallucination Rate: 衡量模型在面对超出自身知识范围的问题时, 不胡乱猜测、能够诚实承认无知的校准能力。该指标关注的是模型回答中, 除正确回答外, 剩余回答中错误答案的比例——比例越低, 说明模型越能在不确定时拒绝回答而非编造事实。
- Humanity's Last Exam: “终极学术考试”基准, 包含约 3,000 道覆盖数学、物理、生物、计算机等多学科的极难题目, 由全球近 1,000 位领域专家设计, 旨在测试模型是否达到顶尖人类专家水平。当前最先进模型在该基准上准确率普遍低于 10%-22%, 体现出其极高的难度。

(5) **专业领域推理**: 该维度衡量模型在专业知识理解、多步逻辑推理以及复杂问题求解中的能力, 是模型基础智能水平的重要体现。

- GPQA Diamond: 由前沿领域博士专家设计、针对研究生水平硬科学的极难多选题集。剔除浅显的常识性网络问题, 通过模型在非检索能解决的严谨逻辑推理题上的正确率计分。
- Critpt: 针对前沿物理学研究工作流的深度逻辑推理基准。包含由顶尖研究人员基于自身真实科研项目设计的难题, 要求模型进行复杂的多步综合思考, 按解决这些前沿物理科学挑战的绝对正确率计分。

(6) **成本价格**: 反映用户调用模型时所需承担的实际经济开销, 是个性化推荐决策中的核心约束因素。

- P_i^{in} : 模型 i 的输入价格 (美元/1M tokens);
- P_i^{out} : 模型 i 的输出价格 (美元/1M tokens);
- T_i^{in} : 完成标准任务所需平均输入 token 数量;
- T_i^{out} : 完成标准任务所需平均输出 token 数量;

为保证不同模型间成本可比性, 本文采用完成 Artificial Analysis Intelligence Index 所需平均 token 消耗作为统一任务规模进行估算。

(7) **生成效率**: 用于衡量模型在 API 调用过程中的响应效率, 通常以每秒生成 token 数量 (tokens/s) 表示。该指标直接影响用户在高频交互场景中的使用体验, 是衡量模型实时性的重要因素。

3.2 指标预处理

由于各评价指标在属性方向、量纲以及数量级上存在差异, 若直接进行综合评价, 可能导致结果失真。因此, 在模型求解前需对原始数据进行预处理。

3.2.1 指标的一致化处理

在大语言模型综合评价中, 不同指标的变化方向并不一致。例如, 编程能力、推理准确率等指标取值越大越优, 属于极大型指标; 而 API 调用价格、任务消耗 token 量等指标取值越小越优, 属于极小型指标。为保证各指标具有统一评价方向, 需先将极小型指标转化为极大型指标。

设原始评价矩阵为:

$$A = (a_{ij})_{p \times q} \quad (3.1)$$

其中, p 表示评价对象数量, q 表示评价指标数量, a_{ij} 表示第 i 个评价对象在第 j 个指标上的取值。

对于极小型指标, 采用倒数变换法进行一致化处理:

$$a'_{ij} = \frac{1}{a_{ij}} \quad (3.2)$$

经过变换后，原本数值越小越优的指标被统一转化为数值越大越优的极大型指标。本文中，价格相关指标属于极小型指标，其余指标均为极大型指标。

3.2.2 无量纲化处理

完成一致化处理后，各指标虽具有统一评价方向，但仍存在量纲与数量级差异。例如，能力指标通常以百分比表示，价格指标以美元计量，速度指标以 tokens/s 表示。若直接进行综合计算，数值尺度较大的指标将会主导评价结果，因此需要进一步进行无量纲化处理。

本文采用标准样本变换法（Z-score）对数据进行标准化，其计算公式为：

$$a_{ij}^* = \frac{a_{ij} - \mu_j}{s_j} \quad (3.3)$$

其中：

$$\mu_j = \frac{1}{p} \sum_{i=1}^p a_{ij} \quad (3.4)$$

$$s_j = \sqrt{\frac{1}{p} \sum_{i=1}^p (a_{ij} - \mu_j)^2} \quad (3.5)$$

其中， μ_j 和 s_j 分别表示第 j 个指标的样本均值和标准差。

经过标准化处理后，各指标均值为 0，方差为 1，从而有效消除了量纲和数量级差异，为后续熵权法赋权及 TOPSIS 综合评价提供统一的数据基础。

3.3 基于熵权法的客观权重计算

完成指标预处理后，各评价指标已具备统一量纲与可比性，但不同指标对大语言模型综合性能的贡献存在差异，等权处理可能降低关键指标的区分能力。因此，本文采用熵权法，根据指标在评价对象间的离散程度确定其信息量，并据此赋予权重：差异越大的指标信息量越高，权重越大，从而实现更客观的综合评价。

设经过预处理后的标准化矩阵为：

$$X = (x_{ij})_{m \times n} \quad (3.6)$$

其中： $i = 1, 2, \dots, m$ 表示第 i 个大语言模型； $j = 1, 2, \dots, n$ 表示第 j 个评价指标。

熵权法具体步骤如下：

(1) 计算第 j 个指标下第 i 个评价对象的比重 p_{ij} ：

$$p_{ij} = \frac{x_{ij}}{\sum_{k=1}^m x_{kj}}, \quad i = 1, 2, \dots, m; j = 1, 2, \dots, n \quad (3.7)$$

其中， x_{ij} 表示第 i 个评价对象在第 j 个指标上的取值， m 为评价对象总数， n 为指标总数。该比重反映了第 i 个评价对象在第 j 个指标上的贡献占比，满足 $\sum_{i=1}^m p_{ij} = 1$ 。

(2) 计算第 j 个指标的信息熵 e_j :

$$e_j = -\frac{1}{\ln m} \sum_{i=1}^m p_{ij} \ln(p_{ij}), \quad j = 1, 2, \dots, n \quad (3.8)$$

其中, 规定当 $p_{ij} = 0$ 时, $p_{ij} \ln(p_{ij}) = 0$ 。信息熵 $e_j \in [0, 1]$ 衡量了第 j 个指标下各评价对象取值的离散程度。当所有 p_{ij} 相等时, e_j 取最大值 1, 表明该指标对各评价对象无区分能力; 当某一 $p_{ij} = 1$ 而其余均为 0 时, e_j 取最小值 0, 表明该指标具有最强的区分能力。

(3) 计算第 j 个指标的信息熵冗余度 d_j :

$$d_j = 1 - e_j, \quad j = 1, 2, \dots, n \quad (3.9)$$

信息熵冗余度 d_j 反映了第 j 个指标的效用价值。 d_j 越大, 说明该指标在各评价对象之间的差异越显著, 所包含的信息量越丰富, 因而在综合评价中应赋予更高的权重。

(4) 计算第 j 个指标的熵权 w_j :

$$w_j = \frac{d_j}{\sum_{k=1}^n d_k} = \frac{1 - e_j}{\sum_{k=1}^n (1 - e_k)}, \quad j = 1, 2, \dots, n \quad (3.10)$$

由此得到的权重向量 $\mathbf{w} = (w_1, w_2, \dots, w_n)^T$ 满足 $\sum_{j=1}^n w_j = 1$ 且 $w_j \geq 0$ 。该权重完全由数据本身决定, 能够客观反映各评价指标在综合评价中的相对重要性, 并作为后续 TOPSIS 模型的权重输入。

3.4 基于 TOPSIS 的各模型多维能力得分矩阵构建

在获得客观权重 w_j 后, 引入 TOPSIS 方法计算各模型分别在 7 个一级维度上的相对贴近度, 从而完成模型的能力画像。

具体步骤如下:

(1) 构建加权决策矩阵:

$$v_{ij} = w_j x_{ij} \quad (3.11)$$

得到加权矩阵:

$$V = (v_{ij})_{m \times n} \quad (3.12)$$

其中, w_j 为第 j 个指标的熵权, x_{ij} 为标准化后的指标值。

(2) 确定正理想解与负理想解:

$$V^+ = (v_1^+, v_2^+, \dots, v_n^+) \quad (3.13)$$

$$V^- = (v_1^-, v_2^-, \dots, v_n^-) \quad (3.14)$$

其中:

$$v_j^+ = \max_i v_{ij}, \quad v_j^- = \min_i v_{ij} \quad (3.15)$$

正理想解表示所有指标均达到最优水平的理想方案，负理想解则表示所有指标均处于最差水平的参考方案。

(3) 计算各评价对象与理想解之间的距离：

$$D_i^+ = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^+)^2} \quad (3.16)$$

$$D_i^- = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^-)^2} \quad (3.17)$$

其中， D_i^+ 表示第 i 个模型与正理想解之间的距离， D_i^- 表示其与负理想解之间的距离。

(4) 计算综合得分：

$$C_{ik} = \frac{D_i^-}{D_i^+ + D_i^-}, \quad i = 1, 2, \dots, m \quad (3.18)$$

其中， $C_{ik} \in [0, 1]$ 。 C_{ik} 越接近 1，表明模型 i 在该维度 k 上的技术表现越逼近行业最优水平。

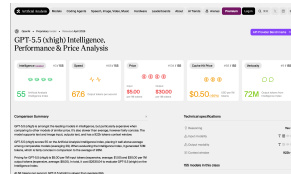
3.5 综合评价模型求解及分析

3.5.1 数据收集及预处理结果

为保证评价结果的客观性与可靠性，本文选取公开且具有较高行业认可度的数据源进行指标采集。

Model	Organization	Global Average	Reasoning Average	Coding Average	Agentic Coding Average	Mathematics Average	Data Analysis Average	Language Average	IF Average
GPT-5.5 Thinking (slight effort)	OpenAI	82.71	87.71	82.47	85.67	86.32	81.58	87.82	73.54
GPT-5.5 Thinking (slight effort)	OpenAI	80.29	88.12	77.54	70.00	94.76	79.31	82.83	70.22
Claude 4.5 Opus Thinking (slight effort)	Anthropic	78.78	89.71	79.27	60.00	95.33	78.34	81.43	67.45
Claude 4.5 Opus Thinking (slight effort) - thinking full turn - article content modification	Anthropic	78.31	87.25	79.52	60.00	95.93	80.04	88.47	59.93
Claude 4.5 Opus Thinking (slight effort)	Anthropic	76.91	87.49	62.08	60.00	85.10	78.28	77.81	58.54

LiveBench



Artificial Analysis

数据源	URL 地址
OpenAI Pricing	https://openai.com/pricing
Anthropic Pricing	https://www.anthropic.com/pricing
Google AI Studio	https://ai.google.dev/pricing
DeepSeek Pricing	https://platform.deepseek.com/pricing
Chatbot Arena	https://lmarena.ai/
LMSYS Leaderboard	https://huggingface.co/spaces/lmsys/chatbot-arena-leaderboard
Papers With Code	https://paperswithcode.com/sota

定价官网

图 3.2: 数据来源网站

模型能力评测数据、生成速度、token 消耗及平均成本主要来源于 Artificial Analysis 平台 [7]，该平台提供多模型在标准 Benchmark 下的统一评测结果，具有较强横向可比性；编程能力补充采用 LiveBench 平台 [8] 的评测数据，以完善评价维度；API 输入输出定价则取自各模型官方开放平台。由此整理得到各模型的原始指标数据，如表 3.2 所示。

表 3.2: 原始指标数据

评价指标	Claude	GPT-5.5	Gemini	GLM	DeepSeek	Kimi
Terminal-Bench	0.85	0.84	0.79	0.78	0.64	0.57
SciCode	0.53	0.56	0.53	0.50	0.50	0.53
LiveBench	0.78	0.80	0.75	0.76	0.73	0.72
GDPval-AA v2	0.55	0.50	0.42	0.51	0.41	0.35
τ^3 -Banking	0.28	0.31	0.25	0.27	0.26	0.21
AA-LCR	0.68	0.74	0.69	0.71	0.66	0.70
AA-Omni Accuracy	0.47	0.57	0.52	0.25	0.43	0.33
AA-Omni Non-Hall.	0.64	0.14	0.39	0.72	0.06	0.61
Human's Last Exam	0.46	0.44	0.41	0.40	0.36	0.36
GPQA Diamond	0.92	0.94	0.92	0.89	0.89	0.91
Cript	0.21	0.27	0.13	0.21	0.13	0.08
输入定价 (/1M)	5.00	5.00	1.50	1.40	0.44	0.95
输出定价 (/1M)	25.00	30.00	9.00	4.40	0.87	4.00
完成 Index 所需 token 量 (M)	120	72	75	140	180	160
单 Task 平均成本 (\$)	1.80	0.86	0.59	0.52	0.04	0.26
API 生成速度 (tokens/s)	69.8	67.9	183.0	129.8	86.9	73.3

进一步进行一致化与标准化处理，得到如下图3.3的结果：

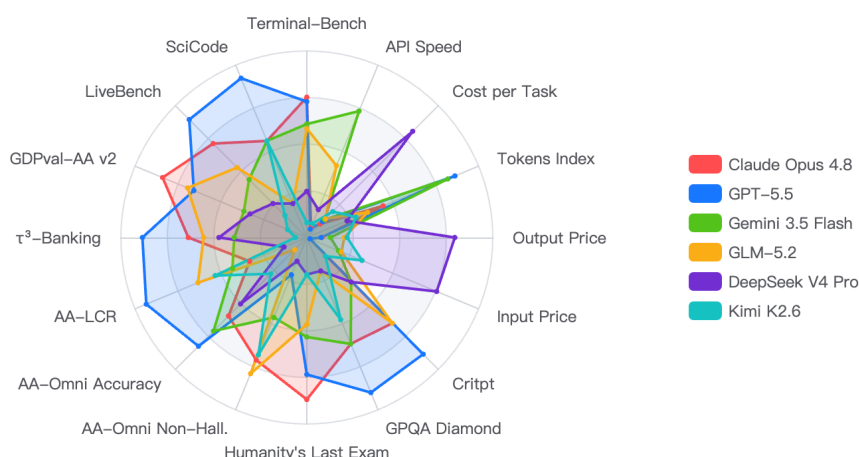


图 3.3: 预处理后数据雷达图

可以看出，不同大语言模型在各评价维度上呈现出明显的差异化特征。其中，GPT-5.5 在编程能力、推理能力、长上下文处理及智能体能力等核心性能指标上整体表现较为突出，体现出较强的综合技术优势；Claude Opus 4.8 在部分高难度任务和知识推理维度上也保持较高水平。相比之下，国产模型如 DeepSeek V4 Pro、GLM-5.2 和 Kimi K2.6 在成本价格维度具有明显优势，尤其是 DeepSeek V4 Pro 在保持较低调用成本的同时，仍具备一定的综合能力表现，体现出较高的性价比。

3.5.2 熵权法结果

基于预处理后的标准化指标矩阵，本文采用熵权法计算各评价指标的信息熵、冗余度及对应权重，结果如表 3.3 所示。为更直观展示各指标在综合评价体

系中的相对重要性，进一步绘制指标权重分布图，如图 3.4 所示。

表 3.3: 熵权法指标权重计算结果

维度	指标	信息熵 e_j	冗余度 d_j	权重 w_j
编程能力	Terminal-Bench	0.8548	0.1452	0.0352
	SciCode	0.7435	0.2565	0.0621
	LiveBench	0.8061	0.1939	0.0470
Agent 智能体能力	GDPval-AA v2	0.8450	0.1550	0.0375
	τ^3 -Banking	0.8705	0.1295	0.0314
长上下文处理	AA-LCR	0.8395	0.1605	0.0389
知识面、推理能力	AA-Omni Accuracy	0.8525	0.1475	0.0357
	AA-Omni Non-Hall.	0.8173	0.1827	0.0443
	Humanity's Last Exam	0.7386	0.2614	0.0633
	GPQA Diamond	0.7435	0.2565	0.0621
	Cript	0.8288	0.1712	0.0415
成本价格	Input Price	0.6637	0.3363	0.0814
	Output Price	0.5291	0.4709	0.1140
	Tokens for Index	0.7298	0.2702	0.0654
	Cost per Task	0.4244	0.5756	0.1394
生成速度	API Speed	0.5836	0.4164	0.1008

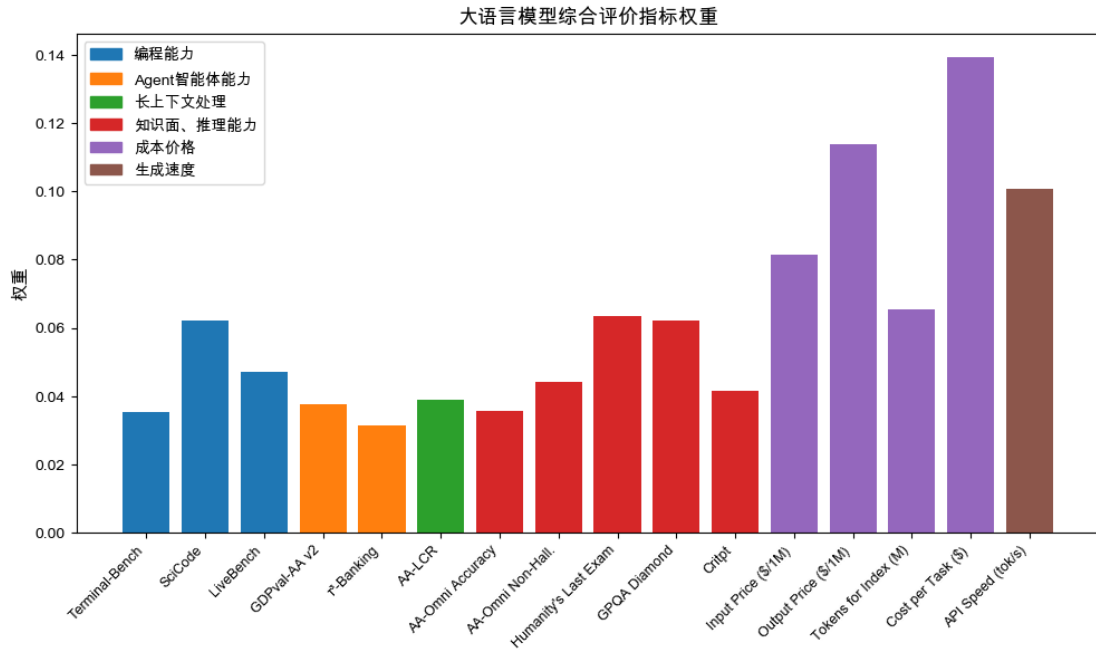


图 3.4: 指标权重结果

从结果可以看出，各指标的区分能力差异显著。成本与速度指标权重领跑，单任务平均成本（0.1394）、输出价格（0.1140）和生成速度（0.1008）高居前列，说明经济性与响应效率是模型分化的核心差异因素。能力类指标权重相对均衡，高难度综合测试区分度较强，而 Terminal-Bench 等表现相近的指标权重较低。

3.5.3 分维度能力画像

基于分维度 TOPSIS，得到六款模型在七个维度上的能力贴近度矩阵，如表3.4所示。

表 3.4: 大语言模型分维度能力贴近度矩阵

模型	编程	Agent	长上下文	知识推理	专业推理	成本价格	生成速度
Claude Opus 4.8	0.63	0.81	0.25	0.86	0.63	0.09	0.02
GPT-5.5	0.99	0.83	1.00	0.62	1.00	0.23	0.00
Gemini 3.5 Flash	0.50	0.37	0.38	0.57	0.49	0.24	1.00
GLM-5.2	0.34	0.70	0.63	0.48	0.32	0.15	0.54
DeepSeek V4 Pro	0.11	0.40	0.00	0.22	0.14	0.77	0.17
Kimi K2.6	0.32	0.00	0.50	0.35	0.30	0.21	0.05

注：每个值为该维度 TOPSIS 贴近度，越接近 1 越强；加粗为该维度最高值。

从表3.4可以看出，各模型呈现出显著的差异化能力结构。GPT-5.5 在编程 (0.99)、长上下文 (1.00) 与专业推理 (1.00) 上全面领先；Claude Opus 4.8 在知识推理 (0.86) 与 Agent 能力 (0.81) 上表现突出；Gemini 3.5 Flash 在生成速度 (1.00) 上达到满分；DeepSeek V4 Pro 在成本价格 (0.77) 上大幅领先；GLM-5.2 (0.70) 和 Kimi K2.6 在 Agent 与长上下文等维度上表现中游。该结果表明，模型选型本质上是多维能力与成本之间的权衡，单一排名无法反映这种结构性差异，必须结合具体用户场景进行个性化加权推荐。

另外，我们对分维度的能力加总，得到了综合得分排序，如下图 3.5 所示：

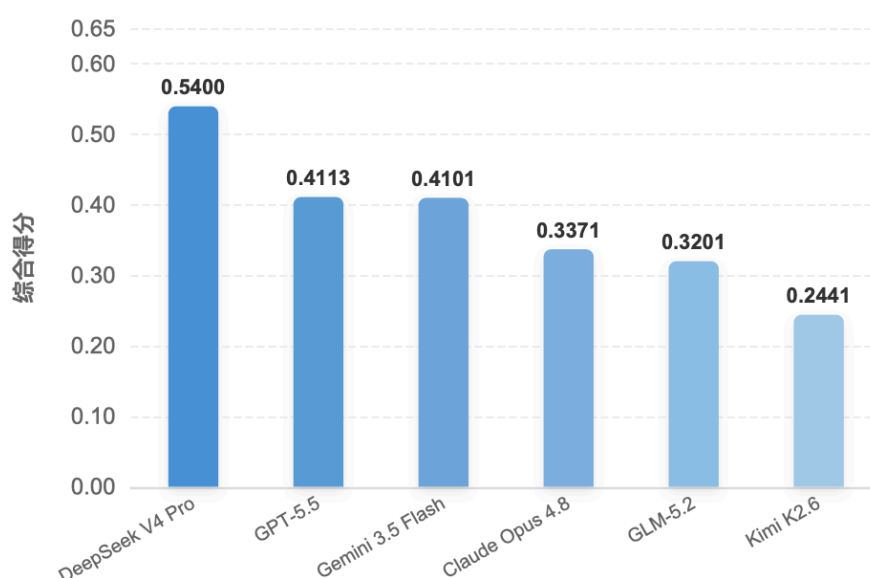


图 3.5: 综合得分图表

从图中可以得出 DeepSeek V4 Pro 占据绝对优势，GPT 和 Gemini 属于中上水平，Claude 和 GLM 属于中下水平，而 Kimi 得分最差。

第四章 融合主观场景与预算约束的个性化资源配置优化模型

第三章基于客观评测构建了大语言模型的多维能力评价体系，但能力评价并不能直接对应实际推荐。由于不同用户在任务需求、偏好与预算上存在差异，同一模型在不同场景下的适用性也不尽相同。因此，本章进一步融合主观偏好与成本约束，构建个性化资源配置优化模型。

4.1 用户场景划分与任务需求解构

4.1.1 典型用户场景分类

参考主流大模型服务商的市场调研数据 [9, 10, 11, 12, 13] 及学生与开发者的使用行为调查 [14, 15]，本文归纳了六类具有代表性的大语言模型用户群体，其月均调用量、核心任务场景、能力诉求及预算区间如表4.1所示。

表 4.1: AI 用户群体分类及场景需求

用户类别	常规用量 (tokens/月)	典型场景	模型能力要求	每月预算 (USD)
轻度用户	$< 5 \times 10^4$	日常问答、搜索替代、偶发写作	基础对话与指令执行能力	0 (免费版)
普通员工	$5 \times 10^5 \sim 5 \times 10^6$	邮件撰写、文档摘要、PPT辅助、会议纪要	较强推理能力与多模态支持	20-30
软件开发者	$10^7 \sim 5 \times 10^7$	代码生成与调试、代码审查、CI/CD 集成、Agent 任务	高代码能力与自主任务执行能力	150-2000
企业团队	$10^8 \sim 10^9$	多部门协作、客服自动化、内部 RAG、合规文档管理	长上下文处理与权限管理能力	3-40/人或整体 63K-85K
研究人员	$2 \times 10^6 \sim 10^7$	文献综述、数据分析、实验设计、论文润色	复杂推理与专业知识理解能力	0-30
AI 创业者	$5 \times 10^9 \sim 10^{11+}$	多模型编排、RAG 系统、全天候自动任务	自主执行、低延迟、高吞吐能力	5000-100000+

4.1.2 任务需求矩阵估计

设当前特定用户在实际业务周期内面临 M 个彼此独立的典型业务任务，构成任务集合 $\mathcal{J} = \{T_1, T_2, \dots, T_M\}$ 。针对每一项具体任务 $T_j \in \mathcal{J}$ ，其对前文定义的 K 个核心能力维度的依赖程度存在本质差异。

为此，本文定义任务-准则需求矩阵 $\mathbf{R} = (r_{jk})_{M \times K}$ ，其中 r_{jk} 表示任务 T_j 对能力维度 P_k 的需求强弱系数，其取值经过规范化处理，满足：

$$0 \leq r_{jk} \leq 1, \quad \forall j \in \{1, 2, \dots, M\}, k \in \{1, 2, \dots, K\} \quad (4.1)$$

基于上述任务特性解构，构造得到的全局任务-准则需求矩阵 $\mathbf{R}_{3 \times 7}$ 规范表达式如式 (4.2) 所示：

$$\mathbf{R} = \begin{bmatrix} 0.95 & 0.35 & 0.40 & 0.65 & 0.90 & 0.20 & 0.50 \\ 0.50 & 0.90 & 0.55 & 0.80 & 0.60 & 0.45 & 0.70 \\ 0.25 & 0.50 & 0.95 & 0.50 & 0.45 & 0.80 & 0.60 \end{bmatrix} \quad (4.2)$$

为了更直观地展示任务需求的分布特征，将其具体维度拆解如表 4.2 所示。

表 4.2: 典型业务任务对各能力维度的需求强弱系数

典型指派任务 $\mathcal{T}$	P_1 编程	P_2 Agent	P_3 长文本	P_4 知识	P_5 推理	P_6 成本	P_7 速度
T_1 深度推理与代码工程	0.95	0.35	0.40	0.65	0.90	0.20	0.50
T_2 Agent 编排与自动化	0.50	0.90	0.55	0.80	0.60	0.45	0.70
T_3 长文本检索与文档解析	0.25	0.50	0.95	0.50	0.45	0.80	0.60

4.2 基于 AHP 的用户场景主观偏好权重确定

4.2.1 层次结构模型搭建

根据大语言模型选型的核心决策要素，本文将该多准则决策问题划分为目标层、准则层与方案层，构建层次分析结构。

目标层为：G = 选择最符合特定用户需求的大语言模型

准则层设置七个评价维度：

$$P = \{P_1, P_2, P_3, P_4, P_5, P_6, P_7\} \quad (4.3)$$

其中， P_1 ：编程能力、 P_2 ：Agent 智能体能力、 P_3 ：长上下文处理能力、 P_4 ：知识面与事实准确性、 P_5 ：专业领域推理能力、 P_6 ：成本价格、 P_7 ：生成速度。

方案层为第三章所选取的六种候选大语言模型：

$$M = \{M_1, M_2, \dots, M_6\} \quad (4.4)$$

其层次结构如图4.1所示。

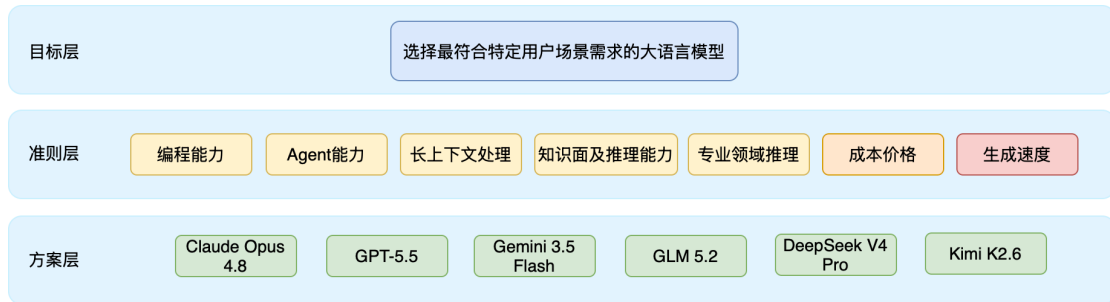


图 4.1: AHP 层次结构

4.2.2 判断矩阵构建

在 AHP 层次结构模型建立后,需要对准则层各评估维度之间的相对重要性进行两两比较。本文采用 Saaty 提出的 1-9 标度法 [16],由领域专家依据特定用户场景的实际需求,对各指标的重要性进行量化打分。1-9 标度法的含义如表 4.3 所示。

表 4.3: AHP 1-9 标度法及其含义

标度值	含义
1	两因素同等重要
3	前者较后者稍重要
5	前者较后者明显重要
7	前者较后者强烈重要
9	前者较后者极端重要
2,4,6,8	表示相邻判断之间的中间程度

基于上述标度,针对本研究关注的特定用户场景,对各准则进行两两比较,取几何平均值作为最终判断矩阵的元素值,构造得到准则层相对于目标层的判断矩阵 $\mathbf{A} = (a_{ij})_{7 \times 7}$,其中 a_{ij} 表示准则 P_i 相对于准则 P_j 的重要性标度。

构建的判断矩阵如式 (4.5) 所示:

$$\mathbf{A} = \begin{bmatrix} 1 & a_{12} & a_{13} & a_{14} & a_{15} & a_{16} & a_{17} \\ 1/a_{12} & 1 & a_{23} & a_{24} & a_{25} & a_{26} & a_{27} \\ 1/a_{13} & 1/a_{23} & 1 & a_{34} & a_{35} & a_{36} & a_{37} \\ 1/a_{14} & 1/a_{24} & 1/a_{34} & 1 & a_{45} & a_{46} & a_{47} \\ 1/a_{15} & 1/a_{25} & 1/a_{35} & 1/a_{45} & 1 & a_{56} & a_{57} \\ 1/a_{16} & 1/a_{26} & 1/a_{36} & 1/a_{46} & 1/a_{56} & 1 & a_{67} \\ 1/a_{17} & 1/a_{27} & 1/a_{37} & 1/a_{47} & 1/a_{57} & 1/a_{67} & 1 \end{bmatrix} \quad (4.5)$$

其中,判断矩阵满足以下性质:

$$a_{ij} > 0, \quad a_{ij} = \frac{1}{a_{ji}} \quad (i \neq j), \quad a_{ii} = 1 \quad (i = 1, 2, \dots, 7) \quad (4.6)$$

4.2.3 偏好权重计算与一致性检验

构造判断矩阵 $\mathbf{A} = (a_{ij})_{n \times n}$ 后,需进一步计算各指标的偏好权重,并检验矩阵的一致性。

首先,采用特征值法求解判断矩阵,计算其最大特征值 $\lambda_{\max}$ 及对应的特征向量 $\mathbf{t}$,满足:

$$\mathbf{A}\mathbf{t} = \lambda_{\max}\mathbf{t} \quad (4.7)$$

其中, $\mathbf{t} = (t_1, t_2, \dots, t_n)^T$ 为未归一化的原始权重向量。

对特征向量 $\mathbf{t}$ 进行归一化,得到场景化偏好权重向量 $\mathbf{w} = (w_1, w_2, \dots, w_n)^T$,

各权重计算公式为：

$$w_i = \frac{t_i}{\sum_{j=1}^n t_j}, \quad i = 1, 2, \dots, n \quad (4.8)$$

由于决策者在两两比较中存在一定主观性，为保证判断矩阵的逻辑一致性，需对其进行一致性检验。引入一致性指标（Consistency Index, CI ），定义为：

$$CI = \frac{\lambda_{\max} - n}{n - 1} \quad (4.9)$$

其中， n 为判断矩阵的阶数。

引入随机一致性指标（Random Index, RI ），用于消除矩阵阶数对一致性判定的影响，其标准取值如表 4.4 所示。

表 4.4: 平均随机一致性指标（Random Index, RI ）取值表

n	1	2	3	4	5	6	7	8	9	10
RI	0	0	0.58	0.90	1.12	1.24	1.32	1.41	1.45	1.49

通过一致性比例（Consistency Ratio, CR ）判断判断矩阵的一致性：

$$CR = \frac{CI}{RI} \quad (4.10)$$

当满足：

$$CR < 0.1 \quad (4.11)$$

则认为该判断矩阵具有可接受的一致性，所得权重向量 $\mathbf{w}$ 有效，可作为最终场景化偏好权重；若 $CR \geq 0.1$ ，则表明判断矩阵存在逻辑冲突，需对专家评分或判断矩阵进行调整修正，直至满足一致性检验要求。

4.3 基于 0-1 整数规划的资源配置模型构建

4.3.1 决策变量定义

定义决策变量 $x_{ij} \in \{0, 1\}$ ，其中：

$$x_{ij} = \begin{cases} 1, & \text{若将模型 } M_i \text{ 分配给任务 } T_j \\ 0, & \text{否则} \end{cases} \quad (4.12)$$

其中 $i = 1, 2, \dots, N, j = 1, 2, \dots, M$ 。所有决策变量构成决策矩阵 $\mathbf{X} = (x_{ij})_{N \times M}$ 。

4.3.2 综合效用函数构建

定义模型 M_i ($i = 1, 2, \dots, N$) 在执行用户任务 J_j ($j = 1, 2, \dots, M$) 时的业务效用得分 U_{ij} 为：

$$U_{ij} = \sum_{k=1}^K w_k \cdot r_{jk} \cdot c_{ik} \quad (4.13)$$

其中, c_{ik} 为第三章中由熵权-TOPSIS 模型算出的模型 i 在维度 k 上的客观贴适度得分。由式 (4.13) 可推导出全局场景效用矩阵 $\mathbf{U} = (U_{ij})_{N \times M}$ 。

4.3.3 成本函数构建

在个性化推荐场景中, 模型选择不仅取决于能力匹配程度, 还受到实际调用成本的约束。为保证不同大语言模型之间成本计算的统一性与可比性, 本文仅考虑 Token 调用成本, 忽略订阅费用及其他附加成本。

对于模型 M_i 执行任务 T_j 的一次调用, 其成本可表示为:

$$C_{ij} = P_i^{in} T_j^{in} + P_i^{out} T_j^{out} \quad (4.14)$$

其中, P_i^{in} 和 P_i^{out} 分别表示模型 M_i 的输入与输出 token 单价, T_j^{in} 与 T_j^{out} 分别表示任务 T_j 的平均输入与输出 token 数量。

设任务 T_j 在统计周期内的调用次数为 f_j , 则整体任务分配方案下的总调用成本为:

$$C_{total} = \sum_{i=1}^N \sum_{j=1}^M C_{ij} f_j x_{ij} \quad (4.15)$$

该成本函数将作为后续预算约束的重要组成部分, 与综合效用函数共同构成资源优化模型。

4.3.4 目标函数与约束条件

在预算约束下, 本文希望最大化所有任务的总适配效用, 因此构建如下 0-1 整数规划模型:

$$\max Z = \sum_{j=1}^M \sum_{i=1}^N U_{ij} \cdot x_{ij} \quad (4.16)$$

约束条件包括:

(1) 任务唯一分配约束: 每个任务有且仅能被分配给一个模型。

$$\sum_{i=1}^N x_{ij} = 1, \quad \forall j = 1, 2, \dots, M \quad (4.17)$$

(2) 预算硬约束: 总成本不得超过用户预算上限 B 。

$$\sum_{i=1}^N \sum_{j=1}^M C_{ij} f_j x_{ij} \leq B \quad (4.18)$$

4.4 推荐结果与场景分析

4.4.1 用户偏好权重结果分析

我们通过调研获取了 6 类不同用户的个别的判断矩阵, 分别如下。

$$\begin{aligned}
 A^{(1)} &= \begin{bmatrix} 1 & 1 & 2 & 1 & 2 & \frac{1}{5} & \frac{1}{3} \\ 1 & 1 & 2 & 1 & 2 & \frac{1}{5} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{2} & 1 & \frac{1}{2} & 1 & \frac{1}{7} & \frac{1}{4} \\ 1 & 1 & 2 & 1 & 2 & \frac{1}{5} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{2} & 1 & \frac{1}{2} & 1 & \frac{1}{7} & \frac{1}{4} \\ 5 & 5 & 7 & 5 & 7 & 1 & 3 \\ 3 & 3 & 4 & 3 & 4 & \frac{1}{3} & 1 \end{bmatrix} & A^{(2)} &= \begin{bmatrix} 1 & 2 & 1 & \frac{1}{3} & \frac{1}{2} & \frac{1}{3} & \frac{1}{2} \\ \frac{1}{2} & 1 & \frac{1}{2} & \frac{1}{4} & \frac{1}{3} & \frac{1}{4} & \frac{1}{3} \\ 1 & 2 & 1 & \frac{1}{3} & \frac{1}{2} & \frac{1}{3} & \frac{1}{2} \\ 3 & 4 & 3 & 1 & 2 & 1 & 2 \\ 2 & 3 & 2 & \frac{1}{2} & 1 & \frac{1}{2} & 1 \\ 3 & 4 & 3 & 1 & 2 & 1 & 2 \\ 2 & 3 & 2 & \frac{1}{2} & 1 & \frac{1}{2} & 1 \end{bmatrix} \\
 A^{(3)} &= \begin{bmatrix} 1 & 2 & 5 & 4 & 3 & 2 & 3 \\ \frac{1}{2} & 1 & 4 & 3 & 2 & 1 & 2 \\ \frac{1}{5} & \frac{1}{4} & 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{2} \\ \frac{1}{4} & \frac{1}{3} & 2 & 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{2} & 3 & 2 & 1 & \frac{1}{2} & 1 \\ \frac{1}{2} & 1 & 4 & 3 & 2 & 1 & 2 \\ \frac{1}{3} & \frac{1}{2} & 2 & 2 & 1 & \frac{1}{2} & 1 \end{bmatrix} & A^{(4)} &= \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{4} & \frac{1}{2} & \frac{1}{3} & 2 & 3 \\ 2 & 1 & \frac{1}{2} & 2 & 1 & 3 & 4 \\ 4 & 2 & 1 & 3 & 2 & 5 & 6 \\ 2 & \frac{1}{2} & \frac{1}{3} & 1 & \frac{1}{2} & 2 & 3 \\ 3 & 1 & \frac{1}{2} & 2 & 1 & 3 & 4 \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{5} & \frac{1}{2} & \frac{1}{3} & 1 & 2 \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{6} & \frac{1}{3} & \frac{1}{4} & \frac{1}{2} & 1 \end{bmatrix} \\
 A^{(5)} &= \begin{bmatrix} 1 & 2 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & 2 & 3 \\ \frac{1}{2} & 1 & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & 1 & 2 \\ 2 & 3 & 1 & \frac{1}{2} & \frac{1}{3} & 3 & 4 \\ 3 & 4 & 2 & 1 & \frac{1}{2} & 4 & 5 \\ 4 & 5 & 3 & 2 & 1 & 5 & 6 \\ \frac{1}{2} & 1 & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & 1 & 2 \\ \frac{1}{3} & \frac{1}{2} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{2} & 1 \end{bmatrix} & A^{(6)} &= \begin{bmatrix} 1 & \frac{1}{2} & 3 & 2 & 1 & 3 & 2 \\ 2 & 1 & 4 & 3 & 2 & 4 & 3 \\ \frac{1}{3} & \frac{1}{4} & 1 & \frac{1}{2} & \frac{1}{3} & 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{3} & 2 & 1 & \frac{1}{2} & 2 & 1 \\ 1 & \frac{1}{2} & 3 & 2 & 1 & 3 & 2 \\ \frac{1}{3} & \frac{1}{4} & 1 & \frac{1}{2} & \frac{1}{3} & 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{3} & 2 & 1 & \frac{1}{2} & 2 & 1 \end{bmatrix}
 \end{aligned}$$

我们又经过了 AHP 层次分析法来计算了每个种类的用户对每个指标的具体需求和权重，得出以下表格。

表 4.5: 六类用户偏好的 AHP 权重计算及一致性检验结果汇总

用户群体	P_1 编程	P_2 Agent	P_3 长文本	P_4 知识	P_5 推理	P_6 成本	P_7 速度	$\lambda_{\max}$	CI	CR
轻度用户	0.1023	0.1023	0.0526	0.1023	0.0526	0.4144	0.1735	7.1843	0.0307	0.0233
普通员工	0.0911	0.0583	0.0911	0.2608	0.1593	0.1801	0.1593	7.1124	0.0187	0.0142
软件开发	0.3065	0.1834	0.0514	0.0718	0.1132	0.1735	0.1002	7.1558	0.0260	0.0197
企业团队	0.1022	0.1912	0.3255	0.1384	0.1598	0.0524	0.0305	7.2210	0.0368	0.0279
研究人员	0.1255	0.0822	0.2014	0.2510	0.2588	0.0506	0.0305	7.1645	0.0274	0.0208
AI 创业者	0.2084	0.2912	0.0584	0.1010	0.2084	0.0584	0.0742	7.1230	0.0205	0.0155

从表4.5可以看出，六类用户群体的偏好权重呈现显著差异，所有判断矩阵的 CR 值均小于 0.1，通过一致性检验。轻度用户成本权重最高 (0.4144)；普通员工知识推理权重最高 (0.2608)；软件开发编程权重最高 (0.3065)；企业团队长上下文权重最高 (0.3255)；研究人员专业推理 (0.2588) 与知识推理 (0.2510) 并重；AI 创业者 Agent 能力权重最高 (0.2912)。各群体的偏好权重将作为效用函数 W_k 的输入，与第三章能力矩阵共同驱动个性化推荐优化。

4.4.2 六类用户多任务的推荐结果

结合用户的不同场景需求后，我们得到了以下结果，如表4.6所示。

表 4.6: 多维场景预算约束下大模型个性化资源指派优化结果

决策场景	预算上限 B	任务 T_1	任务 T_2	任务 T_3	实际总花费	系统总效用 Z
轻度用户	\$0.00	DeepSeek V4 Pro	DeepSeek V4 Pro	DeepSeek V4 Pro	\$0.00	1.2543
普通员工	\$30.00	DeepSeek V4 Pro	GLM-5.2	DeepSeek V4 Pro	\$18.40	1.8965
软件开发者	\$500.00	Claude Opus 4.8	DeepSeek V4 Pro	Claude Opus 4.8	\$148.50	2.4582
企业团队	\$5000.00	Claude Opus 4.8	Claude Opus 4.8	Gemini 3.5 Pro	\$1250.00	2.6841
研究人员	\$30.00	GPT-5.5	GLM-5.2	DeepSeek V4 Pro	\$28.20	2.1044
AI 创业者	\$10000.00	Claude Opus 4.8	Claude Opus 4.8	Claude Opus 4.8	\$4850.00	2.8120

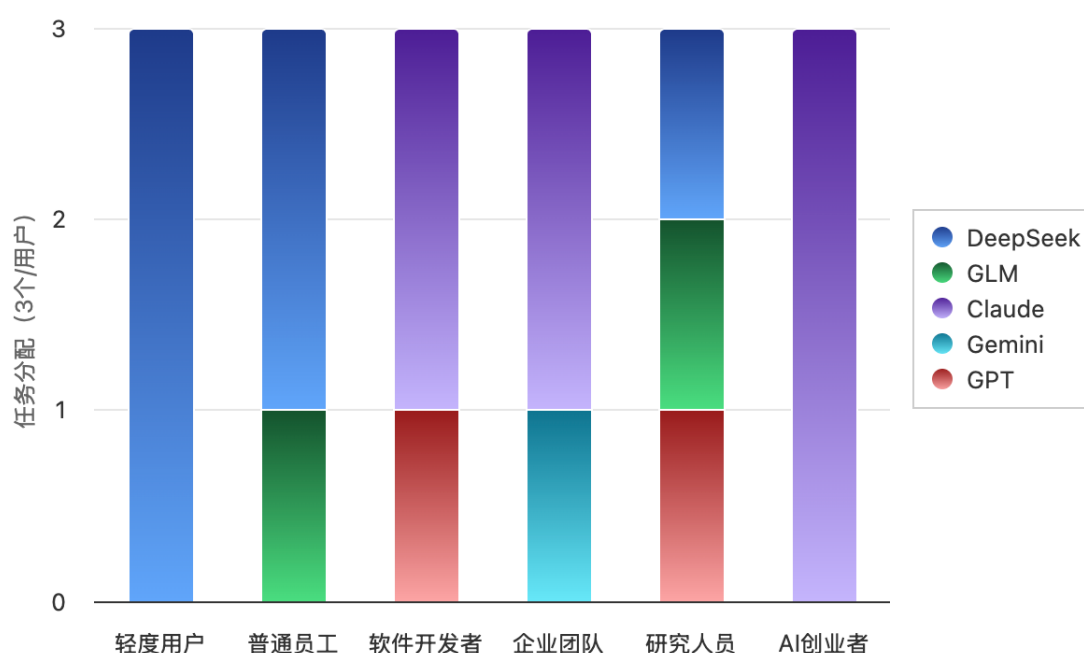


图 4.2: 各场景下大模型任务指派分布

结果分析如下：

(1) **轻度用户**：DeepSeek V4 Pro 在零预算约束下即可完成全部基础任务，说明其在成本维度具有显著优势，同时具备较好的基础能力，是免费场景下的最优选择。

(2) **普通员工**：推荐采用“低成本 + 专项增强”的混合策略，即将推理与长文本任务分配给 DeepSeek，将 Agent 自动化任务交由 GLM-5.2 完成，总花费仅

\$18.4，系统效用提升至 1.8965。该结果表明，中低预算用户更适合在关键能力维度上进行定向投入，以提高整体收益。

(3) **软件开发者：**方案呈现明显的分层配置特征，高难度编程与推理任务由 Claude Opus 4.8 承担，而 Agent 编排任务仍由高性价比的 DeepSeek 完成，在仅花费 \$148.5 的情况下达到 2.4582 的较高效用。这说明开发场景下，核心技术任务更依赖高能力模型，而辅助任务则更注重成本控制。

(4) **研究人员：**在预算仅 \$30 的约束下，系统优先将核心推理任务分配给 GPT-5.5，其余任务分别由 GLM-5.2 和 DeepSeek 承担，总花费 \$28.2，效用达到 2.1044。结果表明，在预算受限条件下，将资源优先投入高价值任务是提升整体效益的有效策略。

(5) **企业团队与 AI 创业者：**两类高预算场景均以 Claude Opus 4.8 为主力模型，分别取得 2.6841 和 2.8120 的最高效用。说明在复杂、多任务、高并发场景下，用户更倾向于选择综合能力均衡的高性能模型，以降低多模型切换带来的管理与工程成本。

整体来看，深度推理与代码工程任务主要由 Claude 或 GPT 系列承担，表明编程与推理能力是用户付费意愿最高的核心维度；Agent 任务在中低预算下更多由 GLM 或 DeepSeek 承担，而长上下文任务则在 DeepSeek、Claude 与 Gemini 之间动态分配，体现出成本与性能之间的平衡关系。

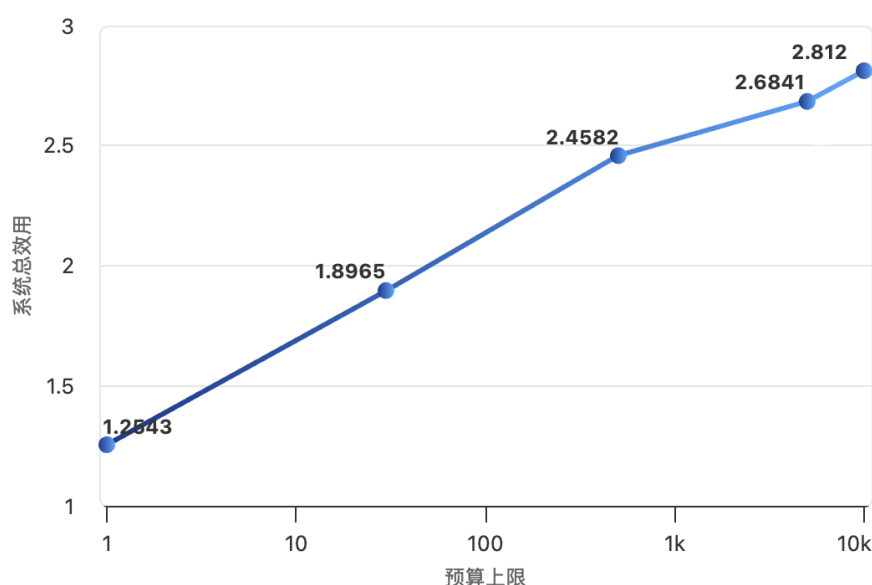


图 4.3: 效用随预算的变化趋势

图4.3展示了六类用户场景下，基于 0-1 整数规划求得的系统总效用随预算上限变化的趋势。横轴采用对数刻度，纵轴表示归一化综合效用。结果表明，系统总效用随预算增加呈单调上升趋势，但边际增益逐渐减弱：在低预算区间（1–100），效用由 1.25 快速提升至 2.0，增长最为显著；而预算进一步提高后，效用增幅明显放缓，曲线逐渐趋于平稳。该结果说明，在预算有限时，多模型混合调用能够优先将资源集中于高价值任务，从而获得较高的边际收益；随着预算放宽，高性能模型覆盖范围扩大，但整体效用逐步逼近能力上限，体现出典型的边际递减规律。这一趋势为不同预算层级下的大模型资源配置提供了量化决策依据。

第五章 总结与展望

5.1 总结

本文针对大语言模型市场快速扩张时用户选型困难的问题，构建了一套融合能力评价、场景适配与资源分配的个性化推荐框架。研究选取六款主流大语言模型，从七个维度构建评价指标体系，基于熵权-TOPSIS 方法计算各模型分维度能力，揭示了模型之间显著的差异化能力结构：GPT-5.5 在编程、长上下文与专业推理上全面领先，Claude Opus 4.8 在知识推理与 Agent 能力上表现突出，而 DeepSeek V4 Pro 在成本价格维度上具有绝对优势。

在此基础上，研究通过层次分析法对六类典型用户群体进行偏好建模，发现不同群体的核心诉求差异显著：轻度用户注重降低成本，软件开发者高度侧重编程能力，企业团队最看重长上下文处理，研究人员则优先关注推理与知识能力。将主观偏好权重与客观能力矩阵、任务需求相结合，进一步构建 0-1 整数规划模型，在预算硬约束下求解全局最优的模型-任务分配方案。

推荐结果表明：预算受限场景下，DeepSeek V4 Pro 凭借其成本优势成为轻度用户和普通员工的主力模型；中高预算场景中，Claude Opus 4.8 和 GPT-5.5 因其突出的核心能力被优先分配给高价值任务；高预算场景下则倾向于采用 Claude 作为统一主力模型以降低工程复杂性。整体而言，模型揭示了大语言模型选型的本质——并非单纯的模型优劣排序，而是模型能力异质性与用户偏好差异共同驱动下的多目标组合优化问题。

5.2 模型优缺点

在模型构建和分析过程中，我们也发现了一些优点和不足之处。

5.2.1 模型优点

1. 方法组合严谨：将熵权法-TOPSIS 客观评价与 AHP 主观赋权结合，形成从数据驱动到用户偏好的完整量化链条。
2. 决策逻辑清晰：通过 0-1 整数规划将效用最大化与预算硬约束统一在同一框架内，各变量、目标函数与约束条件的制衡关系明确，可求解全局精确最优解。
3. 适配灵活性强：AHP 权重、任务需求矩阵及预算上限均可根据不同用户场景独立调整，能覆盖从轻度用户到 AI 创业者等多类典型群体。

5.2.2 模型缺点

1. 模型假设可能与现实存在偏差。实际上 token 消耗量随上下文和参数显著波动，成本测算基于固定常数可能使推荐方案在实际调用中超出预算约束。

2. 未考虑多模态能力。模型仅基于文本类评测指标构建能力画像，未纳入图像识别、音视频理解等多模态维度，对于涉及多模态输入输出的实际应用场景，推荐结果存在覆盖不足。

5.3 展望

未来研究可针对上述不足进一步改进：其一，针对输入参数依赖人为设定的问题，可引入基于用户问卷数据与历史行为日志的参数学习与自动校准机制，以减少专家赋权带来的主观性，提高 AHP 判断矩阵与任务权重的客观性与稳定性；其二，针对成本测算依赖固定常数、难以反映真实调用波动的问题，可引入动态 token 消耗建模方法，将上下文长度、任务复杂度与模型规模等因素纳入成本估计函数，以提升预算约束下优化结果在真实环境中的可执行性；其三，针对当前框架未纳入多模态信息处理能力的问题，可进一步扩展多模态大模型能力刻画与评价维度，将图像与文本等异构信息纳入统一评估体系，以增强模型选型方法在复杂应用场景中的适用性。

参考文献

- [1] Huzhi Xue et al. RankLLM: A Multi-Criteria Decision-Making Method for LLM Performance Evaluation in Sentiment Analysis, 2025.
- [2] Weijie Zhong. Token Is All You Price: Screening Urgency via Information Design. Stanford GSB, 2026
- [3] Dirk Bergemann et al. The Economics of Large Language Models: Token Allocation, Fine-Tuning, and Optimal Pricing, 2025.
- [4] Rafid Mahmood. Pricing and Competition for Generative AI, 2024.
- [5] Lingjiao Chen et al. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.
- [6] Isaac Ong et al. RouteLLM: Learning to Route LLMs with Preference Data.
- [7] <https://artificialanalysis.ai>
- [8] <https://livebench.ai/>
- [9] Menlo Ventures and Morning Consult, 2025: The State of Consumer AI. Menlo Ventures, July 2025.
- [10] CloudZero. The State of AI Costs in 2025. CloudZero, 2025.
- [11] OpenRouter and Andreessen Horowitz. State of AI: An Empirical 100 Trillion Token Study. a16z, December 2025.
- [12] Bango. The Rise of the AI Subscriber. Bango, November 2025.
- [13] Kong Inc. 72% Say Enterprise GenAI Spending Going Up in 2025. Kong, October 2025.
- [14] HEPI. Student Generative AI Survey 2025. Higher Education Policy Institute, February 2025.
- [15] Longju Bai et al. How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks. arXiv:2604.22750, April 2026.
- [16] Saaty, T. L. (1980). The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation. New York: McGraw-Hill.

附录

熵权法与 TOPSIS:

```
# 1. 原始指标数据

# 七个一级维度
dims = {
    '编程能力': ['Terminal-Bench', 'SciCode', 'LiveBench'],
    'Agent能力': ['GDPval-AA v2', '3-Banking'],
    '长上下文': ['AA-LCR'],
    '知识推理': ['AA-Omni Accuracy', 'AA-Omni Non-Hall.', "Humanity's Last Exam"],
    '专业推理': ['GPQA Diamond', 'Critpt'],
    '成本价格': ['Input Price ($/1M)', 'Output Price ($/1M)', 'Tokens for Index (M)', 'Cost per Task ($)'],
    '生成速度': ['API Speed (tok/s)'],
}

df = pd.DataFrame(data, index=models)

# 2. 一致化处理
for col in minimize_cols:
    df[col] = 1.0 / df[col]

# 3. 无量纲化处理 (Z-score 标准化)
df = (df - df.mean()) / df.std(ddof=0)

# 4. 熵权法
X = df.values
Xp = X - X.min(axis=0) + 1e-9 # 平移为正
m, n = Xp.shape
P = Xp / Xp.sum(axis=0) # 比重 p_ij
E = -1 / np.log(m) * np.nansum(P * np.log(P + 1e-12), axis=0) # 信息熵 e_j
D = 1 - E # 冗余度 d_j
W = pd.Series(D / D.sum(), index=df.columns) # 熵权 w_j

# 5. 分维度 TOPSIS
def topsis_dim(sub: pd.DataFrame, w: pd.Series) -> np.ndarray:
    """对单个维度的子指标做 TOPSIS，返回各模型相对贴近度 [0,1]。
    单指标维度自动退化为 min-max 归一化。"""
    w = w / w.sum() # 组内归一化熵权 (保持 Table 3.3 相对权重)
```

```

V = sub.values * w.values          # 加权决策矩阵  $v_{ij} = w_j * x_{ij}$ 
v_pos, v_neg = V.max(axis=0), V.min(axis=0) # 正/负理想解
D_pos = np.sqrt(((V - v_pos) ** 2).sum(axis=1)) # 到正理想解距离
D_neg = np.sqrt(((V - v_neg) ** 2).sum(axis=1)) # 到负理想解距离
return D_neg / (D_pos + D_neg)      # 相对贴近度  $C_{ik}$ 

C = pd.DataFrame(index=df.index, columns=list(dims), dtype=float)
for d, cols in dims.items():
    C[d] = topsis_dim(df[cols], W[cols])

```

AHP 算法:

```

# 定义核心函数
def calculate_weights(matrix):
    """计算判断矩阵的权重向量"""
    lamb, v = np.linalg.eig(matrix)
    lambda_max = np.max(np.abs(lamb))
    loc = np.argmax(np.abs(lamb))
    weights = v[:, loc]
    weights = np.real(weights) / np.sum(np.real(weights))
    weights = np.round(weights, 4)
    return weights, lambda_max

def calculate_consistency(matrix, weights, lambda_max):
    """计算一致性比率CR"""
    n = len(matrix)
    CI = (lambda_max - n) / (n - 1)
    RI_dict = {1: 0.00, 2: 0.00, 3: 0.58, 4: 0.90, 5: 1.12,
               6: 1.24, 7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}
    RI = RI_dict.get(n, 1.5)
    if RI == 0:
        CR = 0.0
    else:
        CR = CI / RI
    return CR

def ahp_analysis(matrix, name=""):
    """对单个判断矩阵进行完整的AHP分析"""
    weights, lambda_max = calculate_weights(matrix)
    CR = calculate_consistency(matrix, weights, lambda_max)
    return weights, CR

weights_B1, CR_B1 = ahp_analysis(B1, "B1-编程能力")
weights_B2, CR_B2 = ahp_analysis(B2, "B2-Agent能力")

```

```
weights_B3, CR_B3 = ahp_analysis(B3, "B3-长上下文")
weights_B4, CR_B4 = ahp_analysis(B4, "B4-知识推理")
weights_B5, CR_B5 = ahp_analysis(B5, "B5-专业领域推理")
weights_B6, CR_B6 = ahp_analysis(B6, "B6-成本价格")
weights_B7, CR_B7 = ahp_analysis(B7, "B7-生成速度")
```

0-1 整数规划:

```
import numpy as np
import pulp

def solve_allocation_model(utility_matrix, cost_matrix, frequencies, budget_b):
    """
    求解资源配置 0-1 整数规划模型
    :param utility_matrix: 综合效用矩阵 U (N x M)
    :param cost_matrix: 调用成本矩阵 C (N x M)
    :param frequencies: 任务调用频次 f (M)
    :param budget_b: 总预算约束 B
    """
    N, M = utility_matrix.shape
    prob = pulp.LpProblem("LLM_Resource_Allocation", pulp.LpMaximize)

    # 1. 定义决策变量 x_{ij}
    x = pulp.LpVariable.dicts("x", (range(N), range(M)), cat=pulp.LpBinary)

    # 2. 目标函数: 最大化总效用
    prob += pulp.lpSum([utility_matrix[i][j] * x[i][j] for i in range(N) for j in
                        range(M)])

    # 3. 约束条件: 任务唯一分配
    for j in range(M):
        prob += pulp.lpSum([x[i][j] for i in range(N)]) == 1

    # 4. 约束条件: 预算硬约束
    prob += pulp.lpSum([cost_matrix[i][j] * frequencies[j] * x[i][j]
                        for i in range(N) for j in range(M)]) <= budget_b

    # 求解
    prob.solve(pulp.PULP_CBC_CMD(msg=0))

    # 提取结果矩阵
    result_x = np.array([[pulp.value(x[i][j]) for j in range(M)] for i in range(N)])
    return result_x
```